Introducción a JavaScript

Desarrollo de Software



¿Qué es JavaScript?

- JavaScript es el lenguaje de programación principal para el desarrollo web. Todos los navegadores usados ampliamente lo ejecutan.
- Tiene un "garbage collector" que se encarga de gestionar la memoria
- Tiene un único hilo de ejecución. Posee un modelo de concurrencia basado en un "loop de eventos" no bloqueante
- Es multiparadigma, tiene soporte para programación orientada a objetos, imperativa y declarativa
- Es "dinámico", sus tipos son implícitos, las variables y funciones pueden ser creadas en tiempo de ejecución y sus valores e incluso sus tipos pueden cambiarse

¿Cómo se ejecuta JavaScript?

- Un runtime de JavaScript se refiere al entorno en el que se ejecuta código JavaScript. Proporciona las estructuras y recursos necesarios para que el código JavaScript se ejecute y se comunique con el sistema subyacente. Uno de los runtimes de JavaScript más conocidos es V8.
- V8 es un motor de ejecución de JavaScript desarrollado por Google. Es de código abierto y se utiliza en varios navegadores web, incluidos Google Chrome y Opera, para ejecutar JavaScript de manera eficiente en el navegador.
- JavaScript se ejecuta tanto de forma interpretada como compilada, dependiendo del entorno en el que se ejecute.

¿Cómo se ejecuta JavaScript?

- En su forma más básica, JavaScript se ejecuta de manera interpretada. Esto significa que el código fuente JavaScript se lee línea por línea y se ejecuta directamente por el motor de JavaScript sin necesidad de una compilación previa.
- Los motores de JavaScript modernos, como V8, utilizan una técnica llamada compilación JIT (just-in-time). En este enfoque, el código JavaScript se analiza y se compila en código de máquina de manera eficiente, pero esta compilación se realiza justo antes de la ejecución.

¿Es posible ejecutar JS fuera del browser?









JavaScript

Tipos de datos y sintaxis

Tipos de datos primitivos

- Los valores primitivos pueden ser de tipo:
  - Undefined
  - Null
  - Boolean
  - Number (64 bit)
  - BigInt
  - String



- Javascript los convierte automáticamente de un tipo a otro (a través de casting implícito).
- Todo otro valor que no sea un valor primitivo es un **objeto**.
- Conceptualmente, **undefined** indica la ausencia de un valor, mientras que **null** indica la ausencia de un objeto.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Data_structures

Variables

- Las variables en JavaScript no son tipadas explícitamente sino que lo toman implícito del valor que se les asigna. Si no se les asigna valor, toman el tipo de dato “undefined”.
- Debido a la naturaleza dinámica del lenguaje las variables pueden almacenar cualquier valor en cualquier momento.
- Los nombres de variables empezar con una letra, un guión bajo `_`, o un signo `$`. Los siguientes caracteres pueden ser dígitos además de los anteriores.
- Maneras de declarar una variable:
  - **var** <- para variables globales y locales (no se recomienda)
  - **let** <- para variables locales
  - **const** <- para constantes locales

https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Grammar_and_types

Strings y Booleans

- **Strings**
  - son una secuencia de caracteres Unicode.
  - Cada carácter puede ser accedido por su índice (el primero es cero).
  - Los strings se encierran entre " ", ' ', o ``
    - Los backticks (3ra opción) permiten definir strings de múltiples líneas e interpolación de strings usando expresiones.

```
2 let str = `Backticks ${rule} when it comes down to template expressions`;
```

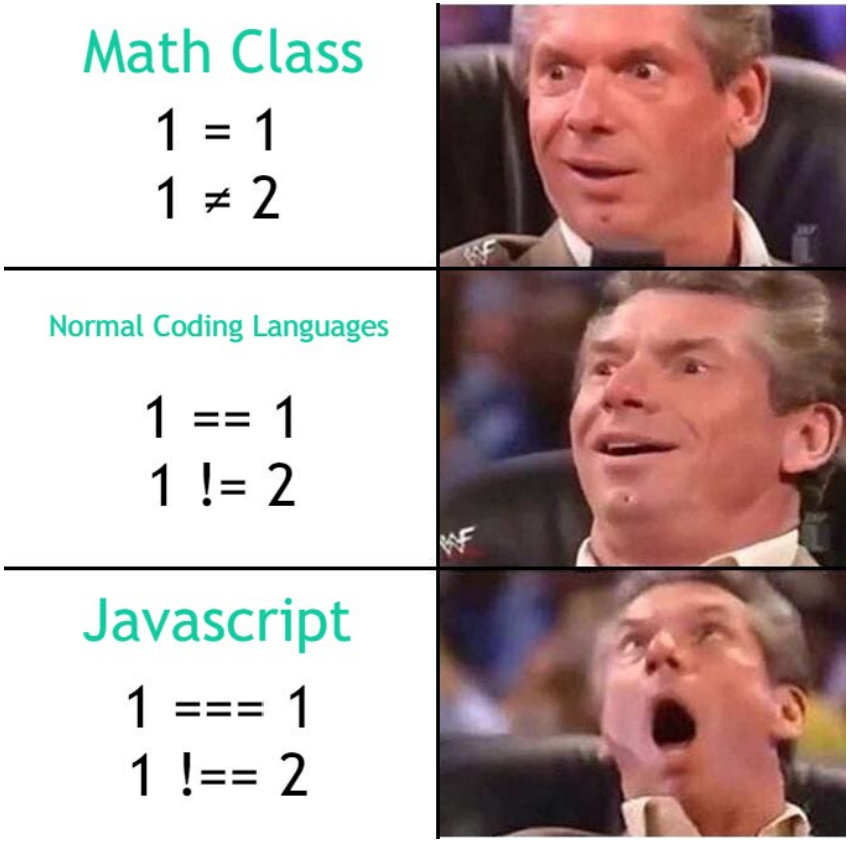
- **Booleans**
  - JavaScript las primitivas Boolean de javascript son **true** y **false**
  - False, 0, string vacio (""), NaN, null, y undefined son interpretados como **falso**.
  - Todo lo demás es interpretado como **verdadero**.

¿Cómo se comparan cosas?

- JavaScript ofrece dos formas para calcular la igualdad, la igualdad estricta o la igualdad débil.
  - La igualdad débil realizará conversión de tipos cuando compara dos valores.
  - En tanto que la forma estricta chequea los valores sin ninguna conversión de tipo. ---> por lo que se recomienda usar la estricta
- La comparación estricta se realiza con **===** (!==) mientras que la débil con **==** (!=)
- JavaScript también provee los operadores **<**, **>**, **<=** y **>=**

¿Cómo se comparan cosas?

- **Comparación estricta ===**
  - Si los valores son de **diferentes tipos de datos** NO son IGUALES.
  - Para valores no numéricos del mismo tipo, son iguales si sus **valores** son los mismos.
  - Para valores numéricos primitivos, compara si **los tipos y los valores son iguales**.



Valores Truthy & Falsy de variables

- Una particularidad de javascript es que una variable al momento de ser evaluada en un “contexto booleano”, se consideran verdaderos o falsos dependiendo de su contenido
- Esta práctica se utiliza mucho en validaciones en los programas de JS y puede resultar poco intuitiva para quienes vengan de lenguajes más tradicionales.
  - Podemos forzar esta evaluación usando un bloque condicional if(variable) o también utilizando una doble negación !!variable.

Operadores

Operator Name	Operator
Arithmetic	+, -, *, /, %, **, ++, --
Assignment	=, +=, -=,*=, /=, %=, **=
String	+
Comparison	<, >, >=,<=, ==, !=
Identity	===
Logical	||, &&, !
Bitwise	&, |, ~, ^, <<, >>,>>>

y más...
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Expressions_and_operators



Ejemplos

```
1   var noDefinida;
2   var variable = 123;
3   const constante = "constante";
4   let variableLocal = '123';
5
6   console.log(variable) // 123
7   console.log(variableLocal == variable)  // true
8   console.log(variableLocal === variable) // false
9
10  console.log("la variable tiene algun valor? ", !!variable); // la variable tiene algun valor? true
11  variable = null
12  console.log("la variable tiene algun valor? ", !!variable); // la variable tiene algun valor? false
13
14  console.log(noDefinida === variable) // false
15  console.log(`variable=${variable}, noDefinida=${noDefinida}`); // variable=null, noDefinida=undefined
16
17  // se puede asignar cualquier cosa a una misma variable
18  variableLocal = 200
19  variableLocal = false
20  varialbeLocal = new BigInt(500)
```

Sintaxis

```
if (condition1) {
    statement1;
} else if (condition2) {
    statement2;
} else if (conditionN) {
    statementN;
} else {
    statementLast;
}
```

```
switch (expression) {
    case label1:
      statements1;
      break;
    case label2:
      statements2;
      break;
    // ...
    default:
      defaultStatements;
}
```

```
for (let i=0; i<100; i++) {
    console.log(i);
}

let x = 0;
while (x < 100) {
    if (x == 32) {
        continue;
    }
    console.log(x);
    x++;
}

x = 0;
do {
    if (x == 32) {
        break;
    }
    console.log(x);
    x++;
} while (x < 100);
```

```
try {
    // execute something that may throw
} catch (exc) {
    // handle the errors
} finally {

}
```

```
const x = "DESO-UTN"
for (letra of x) {
    console.log(letra);
}
```

Sintaxis: funciones

```
1   function suma(a, b) {
2       return a + b;
3   }
4
5   const arrowFunc = (a, b) => a + b;
6
7   const multipleLineFunc = () => {
8       console.log("this is a multiple line arrow function");
9       console.log("very useful :)");
10  };
11
12  // declaring and calling a function in the same line
13  let sqrtDe144 = (num => Math.sqrt(num))(144);
14  console.log(sqrtDe144); // 12
15
16  // you can use functions before declaring them
17  saludar();
18
19  function saludar() {
20      console.log("hola");
21  }
```

FYI: closures
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions#function_scopes_and_closures

Ejemplo

Construir un script de JS que procese un texto de múltiples palabras (el cual será un string) y permita obtener la frecuencia de aparición de las vocales A, E, I, O y U.

Ejemplo: “Esta locuraaa, no la trates de entender”
Resultado: [A: 6, E: 6, I: 0, O: 2, U: 1]

Probar en https://runjs.app/play

Objetos

- Los objetos en JavaScript son similares a los mapas de otros lenguajes de programación, no son más que un conjunto de propiedades que asocian un nombre (o clave) y un valor.
- El valor de una propiedad puede ser una función, en cuyo caso la propiedad es conocida como un método.
- A los objetos que se creen sin depender de una clase en particular, se le pueden agregar propiedades en tiempo de ejecución (ya que no definen un contrato sobre su estructura)

Objetos

```
let persona = {
    nombre: "Bombardino",
    apellido: "Crocodilo",
    edad: 18,
    direccion: {
        calle: "Calle falsa 123",
        ciudad: "Santa Fe"
    },
    saludar: () => console.log(`hola soy ${this.nombre}`)
};

// acceso a sus atributos y metodos
console.log(persona.apellido); // Crocodilo
console.log(persona["edad"]);  // 18
persona.saludar();             // hola soy Bombardino

// a este tipo de objetos le podemos agregar atributos
persona["signo"] = "escorpio";
persona.altura = 180;
console.log(persona.signo);  // escorpio
console.log(persona.altura); // 180

// otra manera de crear objetos
let persona2 = new Object();
persona2.edad = 25;
console.log(persona2.edad); // 25
```

Objetos

```
1   class Persona {
2
3       constructor(nombre, edad) {
4         this.nombre = nombre;
5         this.edad = edad;
6       }
7
8       saludar() {
9         console.log(`Hola, soy ${this.nombre}`);
10      }
11  }
12
13  let persona = new Persona("Juan", 66);
14  persona.saludar(); // Hola, soy Juan
15
16  // a este tipo de objetos
17  // tambien podemos agregarle atributos en runtime
18  persona.apellido = "Pepe";
19  console.log(persona.apellido);
```

Arreglos

- Un arreglo es una **colección ordenada de elementos**. Cada elemento puede ser accedido por un índice numérico.
- Los arreglos en JavaScript no son tipados, y **pueden contener distintos tipos de elementos**. Incluso un elemento puede ser un objeto u otro array.
- Son **dinámicos**, y su tamaño varía según las necesidades, no es necesario indicar su dimensión.
- Los arreglos heredan propiedades de *Array.prototype*, que define un conjunto extenso de métodos útiles para manipularlos

Arreglos

- La forma más simple de crear un arreglo es definir una variables y asignarle una lista extensa de valores separados por coma y encerrados entre corchetes.
  - `var empty = []; // Arreglo vacío`
  - `var primes = [2, 3, 5, 7]; // Arreglo de 5 números`
  - `var misc = [{}, true, "a"];  // Arreglo de distintos tipos de datos`
- También es posible usar el constructor Array()
  - `var a = new Array();`
  - `var a = new Array(5, 4, 3, 2, 1, "testing");`



JAVASCRIPT ARRAY METHODS

```
[1, 2, 3].push(4);                          ⟶  / / [1, 2, 3, 4]
[1, 2, 3].pop();                            ⟶  / / 3
[1, 2, 3].shift();                          ⟶  / /1
[1, 2, 3].unshift(0);                       ⟶  / / [0, 1, 2, 3]
[1, 2, 3].concat([4, 5]);                   ⟶  / / [1, 2, 3, 4, 5]
[1, 2, 3, 4].slice(1, 3);                   ⟶  / / [2, 3]
[1, 2, 3, 4].splice(1, 2);                  ⟶  / / [2, 3]
[1, 2, 3].indexOf(2);                       ⟶  / /1
[1, 2, 3].lastIndexOf(3);                   ⟶  / /2
[1, 2, 3].includes (2);                     ⟶  / / true
[1, 2, 3].join('-');                        ⟶  / / "1-2-3"
[1, 2, 3].reverse();                        ⟶  / / [3, 2, 1]
[1, 2, 3].sort();                           ⟶  / / [1, 2, 3]
[1, 2, 3].map(x => x * 2);                  ⟶  / / [2, 4, 6]
[1, 2, 3].filter(x => x > 1);               ⟶  / / [2, 3]
[1, 2, 3].reduce((a, b) => a + b);          ⟶  / / 6
[1, 2, 3].every(x => x > 0);                ⟶  / / true
[1, 2, 3].some(x => x > 2);                 ⟶  / / true
[1, 2, 3].find(x => x > 2);                 ⟶  / / 3
[1, 2, 3].findIndex(x => x > 2);            ⟶  / /2
```



Ejercicio

Dado un arreglo de estudiantes con la siguiente estructura:

```
const estudiantes = [
    { nombre: "Ana", notas: [8, 7, 9, 8, 12] },
    { nombre: "Luis", notas: [6, 5, 7] },
    { nombre: "María", notas: [10, 9, 10] }
];
```

1. Implementa una **arrow function** que reciba un arreglo de números (notas) y devuelva el promedio.
2. Con esa función, genera un nuevo arreglo llamado **promediosEstudiantes** que contenga objetos con el nombre de cada estudiante y su promedio.
3. Muestra en consola la lista de estudiantes ordenada de mayor a menor promedio (mostrando el nombre y el promedio)
4. Vuelve a resolver lo anterior sin utilizar **while**, **for** ni **do-while**

JavaScript

Conceptos “avanzados”

Desestructuración y el operador Spread

- La **desestructuración** es una expresión de JavaScript que permite “desempacar” valores de arreglos o propiedades de objetos en distintas variables.
- La **sintaxis spread** permite a un elemento iterable tal como un arreglo o cadena sea “expandido” en lugares donde cero o más argumentos (para llamadas de función) o elementos (para Array literales) son esperados, o a un objeto ser expandido en lugares donde cero o más pares de valores clave (para literales Tipo Objeto) son esperados.

Desestructuración y el operador Spread

- La particularidad de la **sintaxis spread** es que la operación termina funcionando como una copia superficial (shallow copy). Lo cual la vuelve útil para agregar elementos a un array u objeto sin modificarlo

```
// Asignando a: 10 y b: 20
const numeros = [10, 20];
let [a, b] = numeros;

console.log(`A: ${a}`)  // Imprime: 10
console.log(`B: ${b}`) // Imprime: 20
```

```
// // Intercambiando los valores de a y b
[a, b] = [b, a];

console.log(`A: ${a}`); // Imprime: 20
console.log(`B: ${b}`); // Imprime: 10
```

Desestructuración y el operador Spread

- Es posible usar el spread operator para quedarse con el resto de valores de un array

```
const muchosNumeros = [10, 20, 30, 40, 50, 60];
const [primero, segundo, ...resto] = muchosNumeros

console.log(primero) // Imprime: 10
console.log(segundo) // Imprime: 20
console.log(resto) // Imprime: [30, 40, 50, 60]
```

Desestructuración y el operador Spread

- Sirve para hacer copias
  - No olvidar que sólo copia valores en el primer nivel (shallow copy)

```
// Copiando un array
const nuevoArray = [...muchosNumeros]

console.log(nuevoArray) // Imprime: [10, 20, 30, 40, 50, 60]
console.log(muchosNumeros) // Imprime: [10, 20, 30, 40, 50, 60]

nuevoArray.pop() // Removemos el último elemento de la copia

console.log(nuevoArray) // Imprime: [10, 20, 30, 40, 50]
console.log(muchosNumeros) // Imprime: [10, 20, 30, 40, 50, 60]
```

Desestructuración y el operador Spread

- Nos permite concatenar delante y detrás de un array

```
// Agregando elementos al principio un array
const numeros = [10, 20];
const masNumeros = [1, ...numeros] // Resultado: [1, 10, 20]

// Agregando elementos al final un array
const numeros = [10, 20];
const masNumeros = [...numeros, 1] // Resultado: [10, 20, 1]

// Concatenando arrays
const array1 = [1, 2];
const array2 = [5, 4];
const array3 = [8, 9, 10];

const resultado = [...array1, ...array3, ...array2]; // Resultado: [1, 2, 8, 9, 10, 5, 4]
```

Desestructuración y el operador Spread

- También pueden utilizarse en objetos:

```
const persona = {
    nombre: "Ricardo",
    apellido: "Sanchez",
    cuil: 20388888885,
    intereses: ["programar en javascript", "quejarse de javascript"],
};

// Copiando un objeto
const personaCopia = { ...persona };

// Extrayendo keys del objeto
const { nombre, intereses } = persona;

console.log(nombre); // Imprime: "Ricardo"
console.log(intereses); // Imprime: ["programar en javascript", "quejarse de javascript"]
```

Desestructuración y el operador Spread

```
// Crear un nuevo objeto modificando keys pero copiando los demas valores
const personaNueva = { ...persona, nombre: "Juan" };

console.log(persona.nombre); // Imprime: "Ricardo"
console.log(persona.apellido); // Imprime: "Sanchez"
console.log(personaNueva.nombre); // Imprime: "Juan"
console.log(personaNueva.apellido); // Imprime: "Sanchez"

// Crear un nuevo objeto agregando keys pero copiando los demás valores
const personaNueva = { ...persona, copado: true };

console.log(!!personaNueva.copado) // Imprime: true
console.log(!!persona.copado) // Imprime: false
```

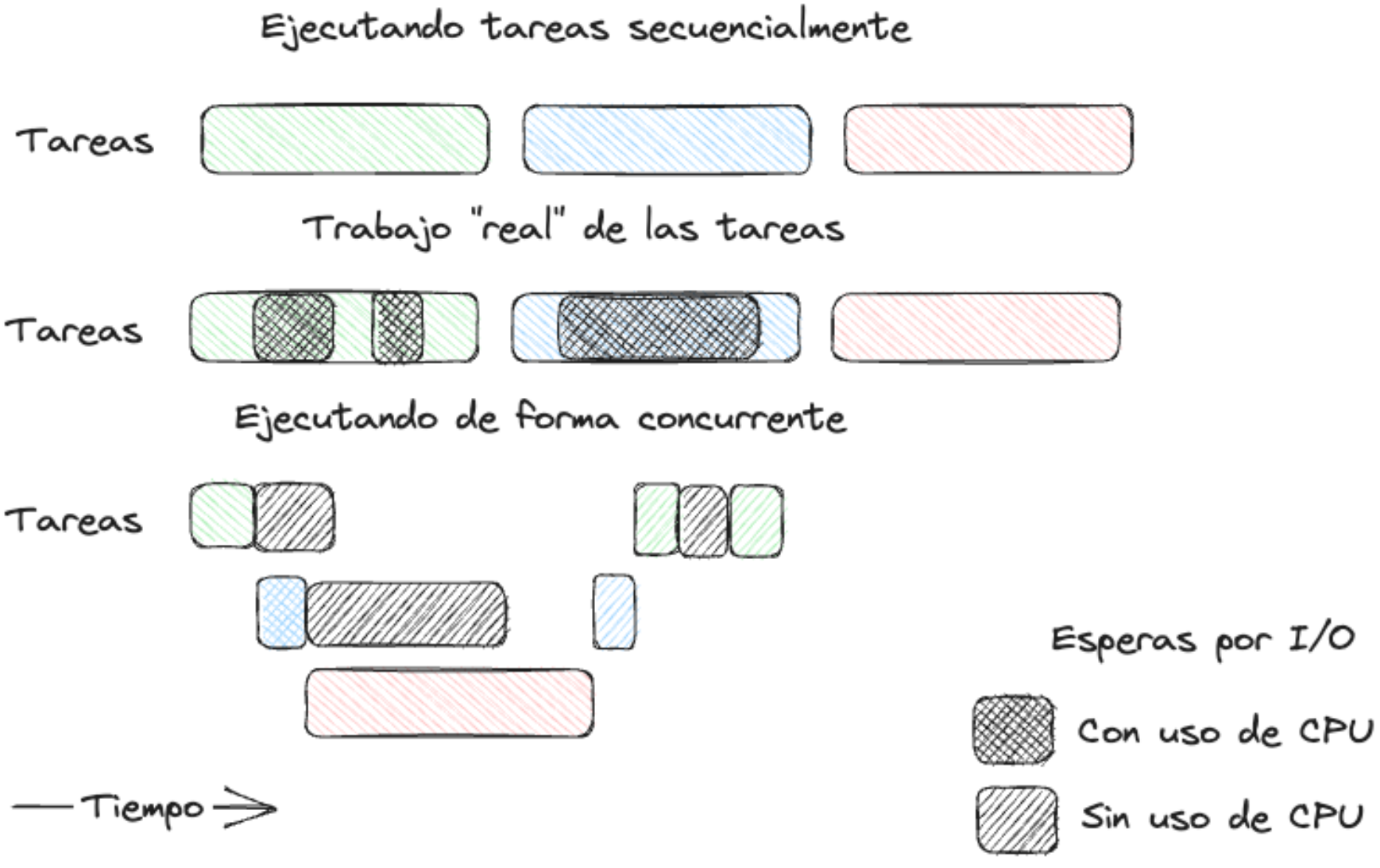
¿Cómo podemos realizar tareas asíncronas?



Asincronía en JavaScript

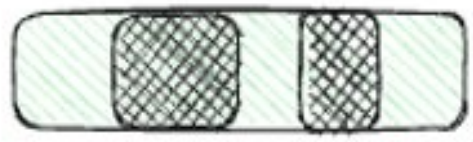
- El runtime de Javascript solo puede ejecutar **una sola cosa a la vez**, es un lenguaje de programación single-threaded (un hilo) y por lo tanto sólo puede procesarse de a una instrucción.
- Esto nos impone una limitante al tratar de realizar operaciones que se extienden en el tiempo como consultar APIs.
- Si nuestra función demora estaremos bloqueando el hilo principal

Asincronía en JavaScript



Asincronía en JavaScript

- ¿Cómo podría verse esta tarea en JS? ---



```
function searchByNameAndMinPrice(name, minPrice) {
  // avg 2 sec to get products
  const products = getProductsByName(name);
  const matchingProductIDs = products
    .filter((p) => p.price >= minPrice)
    .map((p) => p.id);
  // avg 3 sec to get details
  const details = getProductDetails(matchingProductIDs);

  return details;
}
```

Asincronía en Javascript

- ¿Cómo podemos modelar que estas tareas "**demoran**"? Es decir, ¿cómo modelamos esta asincronía de forma que a simple vista entendamos lo que sucede? -> **Promesas**

```
function searchByNameAndMinPrice(name, minPrice) {
  // avg 2 sec to get products
  const products = getProductsByName(name);
  const matchingProductIDs = products
    .filter((p) => p.price >= minPrice)
    .map((p) => p.id);
  // avg 3 sec to get details
  const details = getProductDetails(matchingProductIDs);

  return details;
}
```

Modelando asincronía - Promesas

- En Javascript, una **Promise** es un objeto que PUEDE producir un valor en algún momento futuro. Se utilizan para representar la terminación o el fracaso de una operación asíncrona.
- Admite 3 estados posibles:
  - ✅ **Completada**: La operación terminó exitosamente y tenemos nuestro valor
  - ❌ **Rechazada**: La operación terminó de manera no exitosa y puede contener el error como resultado.
  - ⏳ **Pendiente**: La operación está en proceso, no sabemos si terminará como éxito o error.

Modelando asincronía - Promesas

- El objeto **Promise** expone 2 métodos
  - **then**(value => {}): Recibe como argumento un función que será llamada cuando la promesa se complete con el valor recibido como parámetro.
  - **catch**(err? => {}): Recibe como argumento una función que se llamará en caso de que se rechace y puede tener un argumento con el tipo de error
- Cuando usamos estos argumentos de funciones que se llaman una vez que la tarea es completada o se requiere de alguna acción decimos que estamos usando **callbacks**
- El método **then** aplicado a una función devuelve otra promesa, esto nos permite **encadenarlas**.

Modelando asincronía - Promesas

- Supongamos que tenemos un método **getUserInfo()** que utiliza una API de internet para recuperar los datos de un usuario. Esta tarea no es instantánea, ya que una consulta de red puede demorar varios segundos. Si nuestra función devuelve un objeto promise entonces podemos modelarlo de la siguiente manera:

```
getUserInfo()
  .then(user => console.log(`Hello ${user.name}!`))
  .catch(err => console.error('Error fetching user info:', err));
```

Modelando asincronía - Promesas

- Si existen más funciones que deben ser llamadas, podemos encadenarlas con otro **then()**:
- El método catch al final de nuestro ejemplo se ejecutará tanto si falla cualquiera de las funciones. También se permite encadenar diferentes catch.

```
getUserInfo()
  .then(user => getPendingDebt(user.id))
  .then(debt => console.log(
    debt.amount === 0 ?
      `No pending debt for user ${user.name}. Yay!` :
      `You have a pending debt of $${debt.amount}. Please PAY NOW !!!`
  ))
  .catch(err => console.error('Error fetching user or debt:', err));
```

Modelando asincronía - Promesas

- Desde ES6 existen las palabras claves **async** y **await** que junto a un bloque **try/catch** nos permite expresar el mismo código como:

```
const task = async () => {
  try {
    const user = await getUserInfo();
    const debt = await getPendingDebt(user.id);

    if (debt.amount === 0) {
      console.log(`No pending debt for user ${user.name}. Congrats!`);
    } else {
      console.log(`You have a pending debt of $${debt.amount}. :((`);
    }
  } catch (err) {
    console.error('Error fetching user or debt. Reason:', err);
  }
};

task();
```

Modelando asincronía - Promesas

- La keyword **await** funciona haciendo que, de alguna manera, la ejecución del código “espere” hasta que la promesa se resuelva y luego continúe retornando el resultado de la promesa.
- No se puede utilizar **await** en funciones regulares. Solo en funciones asincrónicas, para esto es necesario que nuestra función incluya la palabra clave **async** en su definición.
- Cuidado! No usar el bloque **finally** de un **try/catch** cuando se utiliza **await** en promesas. En caso de hacerlo veremos como el bloque finally se ejecutará varias veces, lo cual probablemente no sea el resultado esperado


mucho texto

Ejemplo

Implementar una función que reciba un monto en pesos argentinos (ARS) y retorne la cantidad de dólares oficiales a los que equivale, utilizando la cotización del último minuto.
Extra: resolver lo mismo para el valor de dolar blue, en paralelo.

Ejemplo: pesosToUSD(6000) -> (asumiendo que 1 USD = 1400 ARS)
Resultado: 4.28 ARS

Ayuda: Utilizar **fetch({URL})** para consultar a una API

Ejercicio

Implementar un script que consulte el API de los simpsons y:

1. Obtenga los primeros 60 personajes (hay 20 por página)
   a. `https://thesimpsonsapi.com/api/characters?page={pagina}`
2. Para cada personaje, obtenga el nombre del episodio en el cual apareció por primera vez
   a. `https://thesimpsonsapi.com/api/characters/{id}`
3. Y teniendo esta info, imprima el nombre de los episodios (evitando repetidos)

- Resolverlo primero con los métodos que proveen las Promises, y luego utilizando **async** y **await**. Puedes implementar todas las funciones que consideres útiles/necesarias
- https://stackblitz.com/edit/node